

Following one load instruction

A waveform lesson for the IF/ID/EX/MEM/WB RV32I pipeline

Example instruction

```
lw x5, 12(x3)
```

The instruction adds 12 to x3, reads a 32-bit word at that byte address, and writes the word to x5. Another independent instruction can be in a different stage while this load progresses.

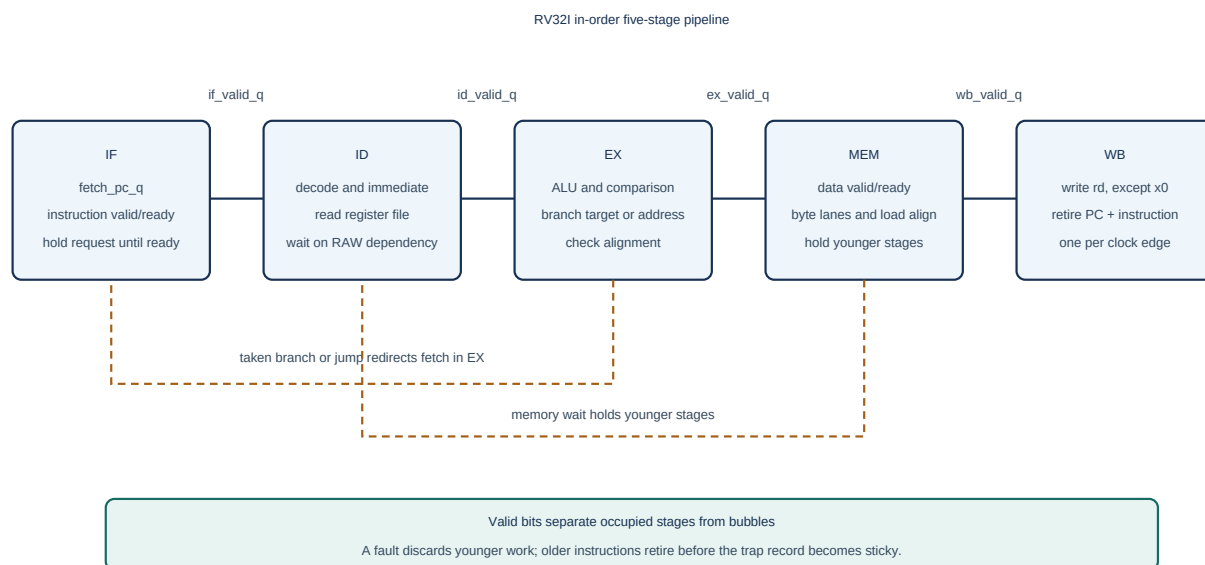


Figure 1. The load crosses IF, ID, EX, MEM, and WB. A valid bit identifies its occupied stage.

Signals to inspect

Signal	What it tells you
if_valid_q / id_valid_q	An accepted instruction and a decoded instruction are present.
ex_valid_q / wb_valid_q	A memory request or a result waiting to retire is present.
dmem_valid_o / dmem_ready_i	The load is requested and then completed.
retire_valid_o	The load has reached architectural retirement.

1. IF and ID

At IF, `imem_addr_o` equals `fetch_pc_q`. `imem_valid_o` stays high until `imem_ready_i` accepts the instruction. The fetched word and its PC enter the IF/ID register, marked by `if_valid_q`.

ID recognizes opcode 0000011 and funct3 010 as LW. It sign extends the I-type immediate 12 and reads `x3`. If an older instruction will write `x3`, dependency holds the load in ID until that write reaches WB. A bubble moves ahead while the load waits.

2. EX

The ALU adds the `x3` value to 12. For LW, address bits [1:0] must be 00. A misaligned address creates a load-address-misaligned fault before any data request. Otherwise the effective address and load controls enter EX/MEM.

3. MEM

`dmem_valid_o` stays high with a stable `dmem_addr_o` while `dmem_ready_i` is low. EX/MEM and younger stages wait. When ready rises, the returned 32-bit word is selected; `dmem_error_i` instead records a load access fault.

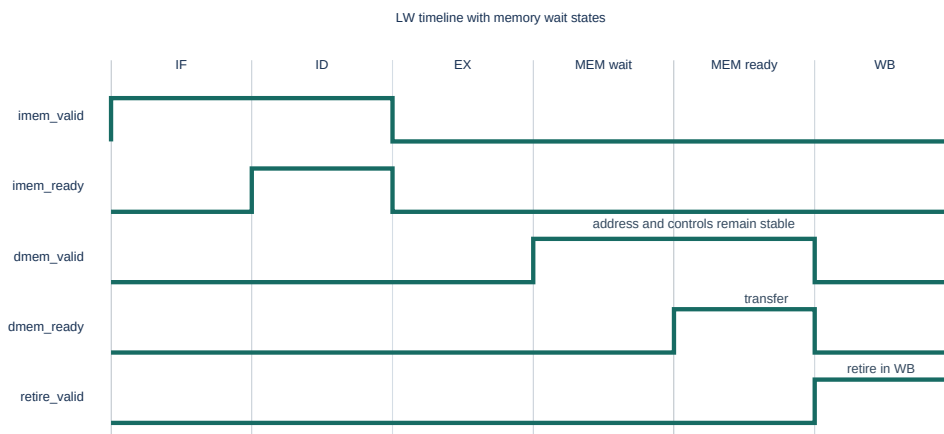


Figure 2. A held load request; stage occupancy can overlap with older or younger work.

4. WB and retirement

The completed load enters MEM/WB. On its WB edge, the register file writes x5 and retire_valid_o pulses with the load's PC and instruction word. A faulting load does neither. x0 remains zero even if selected as the destination.

Try a dependent instruction

Place ADD x6,x5,x5 after the load. Because this baseline has no forwarding, the ADD stays in ID until the load writes x5. Then it reads the new word twice. In the waveform, look for the held IF/ID instruction, a bubble ahead of it, and a later retirement pulse.

Try a byte load

Change LW to LB and the offset to 13. A byte address may have nonzero low bits. Memory still returns the aligned 32-bit word; the core selects the addressed byte and sign extends bit 7. LBU would zero extend that same byte.

Reproduce the waveform

Run `make test-cocotb-waves PYTHON=.venv/bin/python`. Open `sim/build/cocotb/dump.fst` with `waves/rv32i_core.gtkw`. The scoreboard and assertions determine pass/fail; the waveform lets you see why the pipeline stalled or advanced.